

Falcon-SR: a screen-refine algorithm for latent log-abundance correlation networks in compositional sequencing data

Jun Rao^{1,*} and Xiao Liang¹

¹School of Mathematical Sciences, [Institution], [Street], [Postcode], [State], [Country]

*Corresponding author. corresponding-author@example.org

Abstract

Motivation: Microbiome correlation networks are commonly estimated with SparCC for a single composition and SparXCC Case-C for paired compositions, because both methods target Pearson correlations on the latent log-abundance scale that compositional data render identifiable. Existing implementations stop at point estimates: every iterative exclusion round re-solves a dense linear system over all feature pairs, and significance, prior integration, and sparse output have to be bolted on by the user.

Results: We present Falcon-SR, a screen-refine pipeline that is, to our knowledge, the first compositional correlation estimator to combine (i) the SparCC and SparXCC Case-C estimands; (ii) a sparse top- k candidate output that reduces stored edges by an order of magnitude; (iii) closed-form integration of optional signed biological priors; and (iv) approximate permutation calibration delivering candidate-set q -values, all in one open-source implementation. On the feasibility benchmark, Falcon-SR reproduces SparXCC iter on every cross-domain cell at edge overlap ≥ 0.984 and sign accuracy 1.000, and reproduces the SparCC closed-form ranking at edge overlap 1.000 on every single-domain cell where SparCC itself is informative. Against an eight-method baseline panel including Pearson(raw), Pearson(CLR), SPIEC-EASI MB, SPIEC-EASI glasso, and SPIEC-EASI cross-glasso, Falcon-SR is the only sparse method that preserves the SparCC ranks; SPIEC-EASI partial-correlation variants overlap the SparCC reference at only ≈ 0.56 , and SPIEC-EASI cross-glasso collapses to AUROC 0.50 on the hardest cross cell where Falcon-SR fast holds 0.87 and the prior pipeline reaches 0.93 (a +0.06 gain that no other method in the panel can deliver). The prior pipeline raises candidate recall from 0.756 to 0.873 on the same cell, with no regression on easier cells.

Availability and implementation: <https://github.com/JustinRaoV/FALCONE>. Implemented in Python 3.12 with NumPy and SciPy. The repository ships the simulators, baselines, and benchmark runners that produced every numerical claim in this paper.

Contact: corresponding-author@example.org

Supplementary information: Supplementary data are available at Bioinformatics online.

Keywords compositional data, microbiome, correlation network, SparCC, SparXCC, screen-refine

Introduction

Sequencing-derived microbiome tables (16S, ITS, shotgun metagenomic) encode pairwise dependencies only up to the unobserved total abundance per sample. Treating the resulting proportions as Euclidean coordinates inflates spurious negative correlations and suppresses true positive ones (Aitchison, 1986; Lovell et al., 2015). SparCC (Friedman and Alm, 2012) resolved this by estimating the Pearson correlation of the latent log-absolute-abundance vectors through an iterative refinement of the Aitchison variation matrix. SparXCC (Jensen et al., 2024) extended the same idea to two independently normalised compositions via a Case-C double-centred estimator, making bacteria-fungi or phage-host network inference tractable on real cohorts.

The bottleneck for both estimators is the work scope of refinement. Each iterative exclusion round re-solves a dense linear system that

couples every pair of features, and bootstrap-based significance testing multiplies that cost by a hundred or more. FastSpar (Watts et al., 2019) cut the constant factor by re-implementing the loop in optimised C++, and fastCCLasso (Zhang et al., 2024) reduced the per-iteration cost in a related L_1 -regularised estimator, but neither changed the order of work. Methods that change the estimand answer a different scientific question and cannot substitute for SparCC when latent log-abundance correlation is the target; this group includes graphical-lasso variants for partial correlations (Kurtz et al., 2015), proportionality scores (Lovell et al., 2015), and precision-matrix estimators. Brunner et al. (2024) also showed that naively concatenating two normalised compositions distorts the very quantity SparXCC Case-C was designed to recover, ruling out the simplest workaround for multi-domain studies.

No published estimator in this family is sparse and calibrated and prior-aware at the same time. SparCC and SparXCC give dense point

estimates and require bootstrap for significance. FastSpar (Watts et al., 2019) and fastCCLasso (Zhang et al., 2024) preserve the estimand but keep the dense output. SPIEC-EASI variants (Kurtz et al., 2015) are sparse but estimate a different quantity (conditional dependence), so their edges cannot be compared against SparCC's marginal-correlation rankings. The biological priors that microbiome workflows rely on, such as CRISPR spacer matches for phage-host pairs, have no native place in any of these pipelines.

Falcon-SR closes that gap. We design a screen-refine pipeline that preserves the SparCC and SparXCC Case-C estimands, returns a top- k sparse edge table (typically an order of magnitude smaller than the dense $p \times p$ output), integrates optional signed biological priors through a closed-form post-hoc shrinkage, and produces candidate-set q -values from a permutation calibration that runs inside the feasibility wall-clock budget. Each component reduces to a published reference in its limit case, so the design adds capability without sacrificing estimand fidelity. We validate every claim on a controlled simulation grid against an eight-method single-domain baseline panel (SparCC, Pearson(CLR), Pearson(raw), SPIEC-EASI MB, SPIEC-EASI glasso, plus Falcon-SR strict, fast, and fast + calibrate) and a six-method cross-domain panel (SparXCC base, SparXCC iter, SPIEC-EASI cross-glasso, plus Falcon-SR cross fast, fast + calibrate, and fast + prior). All inputs, implementations, and acceptance gates are open for inspection.

Methods

Estimand and scope

For a single composition with basis abundances w_i , Falcon-SR targets $\rho_{ij} = \text{Corr}(\log w_i, \log w_j)$. For two independently normalised compositions with basis abundances w^X and w^Y , it targets $\rho_{ik} = \text{Corr}(\log w_i^X, \log w_k^Y)$. This is the same estimand as SparCC and SparXCC Case-C and is strictly distinct from the proportionality metric ρ_p , partial correlations, or precision-matrix entries; the paper does not use those methods as substitutes.

Preprocessing

The input is a non-negative count matrix $X \in \mathbb{Z}_{\geq 0}^{n \times p}$. Falcon-SR drops features whose summed prevalence falls below a configurable threshold (default ≥ 1 non-zero sample), normalises each row to unit sum, and replaces zeros with the Martin-Fernández multiplicative substitution $\delta = 0.65/p^2$. The routine returns the filtered composition, its log-transform, and the indices of retained features; downstream SparCC and SparXCC comparisons are run on the same filtered matrix so that no method is implicitly evaluated on a different feature set.

Single-domain base score

The Aitchison variation matrix is computed in one BLAS call followed by a vectorised broadcast:

$$t_{ij} = \omega_i^2 + \omega_j^2 - 2\omega_i\omega_j\rho_{ij}, \quad (1)$$

where the basis variances solve in closed form as $\omega_i^2 = (b_i - S)/(p-2)$ with $b_i = \sum_{j \neq i} t_{ij}$ and $S = \sum_i b_i/[2(p-1)]$, and the base correlation follows from $\hat{\rho}_{ij} = (\omega_i^2 + \omega_j^2 - t_{ij})/(2\omega_i\omega_j)$. This estimator is algebraically identical to one SparCC iteration without exclusions

and agrees with the SparCC reference implementation to 10^{-10} on small inputs (`tests/test_single_base.py`).

Single-domain screening and refinement

The screen retains a symmetric top- k union: for each feature i , the k partners with largest $|\hat{\rho}_{ij}|$ are added to the candidate set. An optional `min_abs_score` threshold can promote further pairs above a configured magnitude. Refinement then proceeds one exclusion at a time. At each round, the strongest unexcluded candidate is removed from the basis-variance linear system and the system is resolved on the sparse complement using a SciPy `LinearOperator + cg` matvec; this never materialises the dense $p \times p$ modifier. The loop exits when no remaining candidate exceeds `exclusion_threshold` or after `max_exclusions` rounds.

Cross-domain base score

For paired compositions X and Y , the within-domain SparCC basis standard deviations α and β are estimated separately on each side. The SparXCC Case-C double-centred estimator is then

$$\hat{\rho}_{ik}^{(\text{base})} = \frac{(H_p \text{cov}(\log x, \log y) H_q^\top)_{ik}}{\alpha_i \beta_k}, \quad (2)$$

with centring matrices $H_p = I_p - \frac{1}{p}\mathbf{1}\mathbf{1}^\top$ and H_q defined analogously. The implementation matches the SparXCC base reference to 10^{-10} on small inputs (`tests/test_cross_base.py`).

Cross-domain sparse refinement

Cross-domain refinement uses *edge-driven feature pruning*: each excluded candidate (i, k) drops X-row i and Y-column k from the centring pool. Let $S \subseteq [p]$ and $T \subseteq [q]$ be the surviving feature sets at a given round. The refined estimator is

$$\hat{\rho}_{ik}^{(E)} = \frac{(H_p^{(S)} \text{cov} H_q^{(T)\top})_{ik}}{\alpha_i \beta_k}, \quad (3)$$

where $H_p^{(S)}$ centres only rows in S and $H_q^{(T)}$ only columns in T . If $|S| < 3$ or $|T| < 3$ the centring reverts to the full pool and the diagnostics field `fallback_to_base_centering` is set. Two limits anchor the estimator: with $E = \emptyset$ it reduces to SparXCC base, and when the candidate set covers every pair under the SparXCC iter exclusion rule it recovers the published iterative estimate.

Adaptive growth

The screen budget is adaptive rather than fixed. Falcon-SR runs the screen-refine pipeline at `top_k`, repeats it at $2 \cdot \text{top}_k$, and compares the resulting top-edge sets for Jaccard overlap and sign agreement. The smaller budget is accepted when both metrics exceed `stability_threshold`; otherwise the budget doubles and the comparison repeats, up to `max_top_k`. On reaching that cap without convergence, a human-readable `fallback_reason` string is written into the diagnostics so that downstream code can detect the failure rather than silently trust an unstable result.

Signed biological priors

A `PriorEdge` record carries source/target feature indices, an expected sign $s \in \{-1, 0, +1\}$, a confidence $c \in [0, 1]$, and a free-text provenance string. When the prior weight $\lambda > 0$, the prior pair is

injected into the candidate set even if the statistical screen would have omitted it. After refinement, the candidate edge score is replaced by the analytic minimiser of the penalised loss

$$(\rho - \hat{\rho}_{\text{data}})^2 + \lambda c(\rho - s \cdot \tau)^2, \quad \rho^* = \frac{\hat{\rho}_{\text{data}} + \lambda c s \tau}{1 + \lambda c}, \quad (4)$$

where τ is the configured target magnitude. Two properties of the closed form matter operationally. First, the formula collapses to $\hat{\rho}_{\text{data}}$ exactly when $\lambda = 0$, so existing call sites cannot trigger prior side-effects by accident. Second, the diagnostics field records how many candidate edges disagreed with the prior so that wrong-sign data is visible rather than silently overridden.

Permutation calibration

Calibration permutes the log composition independently per column in the single-domain path, and shuffles Y rows relative to X in the cross-domain path. Each permutation recomputes only the closed-form base score (Equations 1 and 2); the sparse refinement is intentionally skipped, and the maximum $|\hat{\rho}|$ over the candidate set is recorded as a family-wise null statistic M_r . The edge p -value for refined score $\hat{\rho}_e^{(\text{refined})}$ is

$$\hat{p}_e = \frac{1 + \#\{r : M_r \geq |\hat{\rho}_e^{(\text{refined})}|\}}{1 + R}, \quad (5)$$

with R the number of permutations; q -values follow Benjamini-Hochberg. The diagnostics field `calibration_method` carries the value `permutation_base_only` so that downstream code is not free to treat the result as calibration-tight. The trade-off this approximation makes is discussed in Section 4.3.

Results

The experimental design follows the 2026-06-02 execution-design specification (committed under `docs/superpowers/specs/` in the repository). All cells ran with three replicates on the same laptop (Apple Silicon, Accelerate BLAS) using the simulators and baselines shipped with the repository, and per-cell rows are written to `data/falcon_sr*_feasibility.csv`.

Equivalence anchors

We first verified that Falcon-SR reproduces its target estimators exactly in the limits that define them. On randomly sampled count matrices, the single-domain base score matched the SparCC closed form to 10^{-10} , and the cross-domain base score matched SparXCC base to 10^{-10} . The cross-domain sparse refine reduced to SparXCC base when no edges were excluded, and recovered SparXCC iter when the candidate set covered all pairs under the matched exclusion rule. Strict-mode single-domain inference (`mode="strict"`) agreed with the SparCC closed form to four decimal places at every cell where neither estimator hit a numerical fallback.

Single-domain feasibility

We swept $n \in \{100, 500\}$, $p \in \{100, 500, 1000\}$, and `top.k` $\in \{10, 25, 50\}$ with edge density 0.02, and ran every cell through eight estimators: Falcon-SR strict / fast / fast + calibrate, SparCC, Pearson(CLR), Pearson(raw), SPIEC-EASI MB, and SPIEC-EASI glasso. The first five target the latent log-abundance

Pearson correlation; the last three target a different estimand (raw compositional Pearson and partial / inverse-covariance edges) and are included as adjacent context.

Figure 1b reports the headline differentiator. On the $n \geq 500$ cells, all three Falcon-SR variants reach 1.00 edge overlap with the SparCC reference, tying SparCC's own self-overlap and clearing the 0.95 spec gate by a wide margin; Pearson(CLR) follows at 0.98. Pearson(raw) collapses to 0.18, recovering the closure failure of naive Pearson on compositions as a sanity check. SPIEC-EASI MB and glasso reach only 0.56 because their partial-correlation estimand disagrees with SparCC's marginal ranking. The bar plot makes Falcon-SR's position direct: it is the only sparse-output method in the panel whose retained edges coincide with the SparCC ranking.

Figure 1c shows the per-cell AUROC against planted truth. Falcon-SR strict reproduces SparCC exactly because the two share the closed-form basis-variance solve. Falcon-SR fast lags SparCC by ≤ 0.03 AUROC on the same cells, which is the expected cost of a sparse output: AUROC is computed over every pair in the upper triangle, so a method that returns $k \cdot p$ candidate scores instead of $p \cdot (p-1)/2$ scores is implicitly scored against zeros on the non-candidate pairs. Edge overlap (Figure 1b) is the sparse-aware comparison, and Falcon-SR fast reaches 1.00 there. In the $n = 100, p = 1000$ corner of the grid, every latent-correlation estimator collapsed to AUROC 0.50–0.58 against the planted truth, and SparCC and Falcon-SR strict still agreed to four decimal places: the gap reflects underdetermination, not implementation.

Figure 1d records wall-clock against p . Falcon-SR fast and strict sit one order of magnitude above SparCC's closed-form GEMM at $p \leq 1000$, and an order of magnitude below SPIEC-EASI's 50–120 $\times$ overhead (5.1 s for MB and 12.1 s for glasso, against 0.099 s for Falcon-SR fast at $p = 1000$). The permutation-calibrated variant absorbs a further 20 $\times$ at $p = 1000$, which is the cost of producing q -values; SparCC's bootstrap pipeline would multiply its own time by the same factor. The full panel positions Falcon-SR as the second-fastest method in the panel and the fastest method that ships calibration, sparse output, and prior integration together.

Cross-domain feasibility

We swept $n \in \{100, 500\}$, $(p, q) \in \{(100, 100), (500, 500)\}$, and `top.k` $\in \{10, 25\}$ at bipartite edge density 0.01. Each cell was run through six estimators: Falcon-SR cross fast / fast + calibrate / fast + prior, SparXCC base, SparXCC iter, and SPIEC-EASI cross-glasso. The first five target the Case-C cross-correlation estimand; SPIEC-EASI cross-glasso targets the partial cross-correlation on stacked compositions and is included as adjacent context.

Falcon-SR cross fast reached edge overlap 0.99 against the SparXCC iter reference averaged across cells, tying Falcon-SR + calibrate at 0.99 and trailing the SparXCC base/iter pair at 1.00 (Figure 2b). SPIEC-EASI cross-glasso reached only 0.74 averaged, and collapsed to edge overlap 0.006 on the hardest cell ($n = 100$, $p = q = 500$, `top.k` = 10), recovering the cross-domain analogue of the closure failure documented by Brunner et al. (2024).

Per-cell AUROC against planted truth (Figure 2c) exceeded 0.95 for every Falcon-SR variant on every cell except the $n = 100, p = q = 500$ pair, where Falcon-SR fast trails SparXCC iter by ≤ 0.08 AUROC. As in the single-domain panel, the gap is the cost of returning a sparse output under a metric that scores every pair in the upper rectangle: Falcon-SR ranks only its candidate set, and

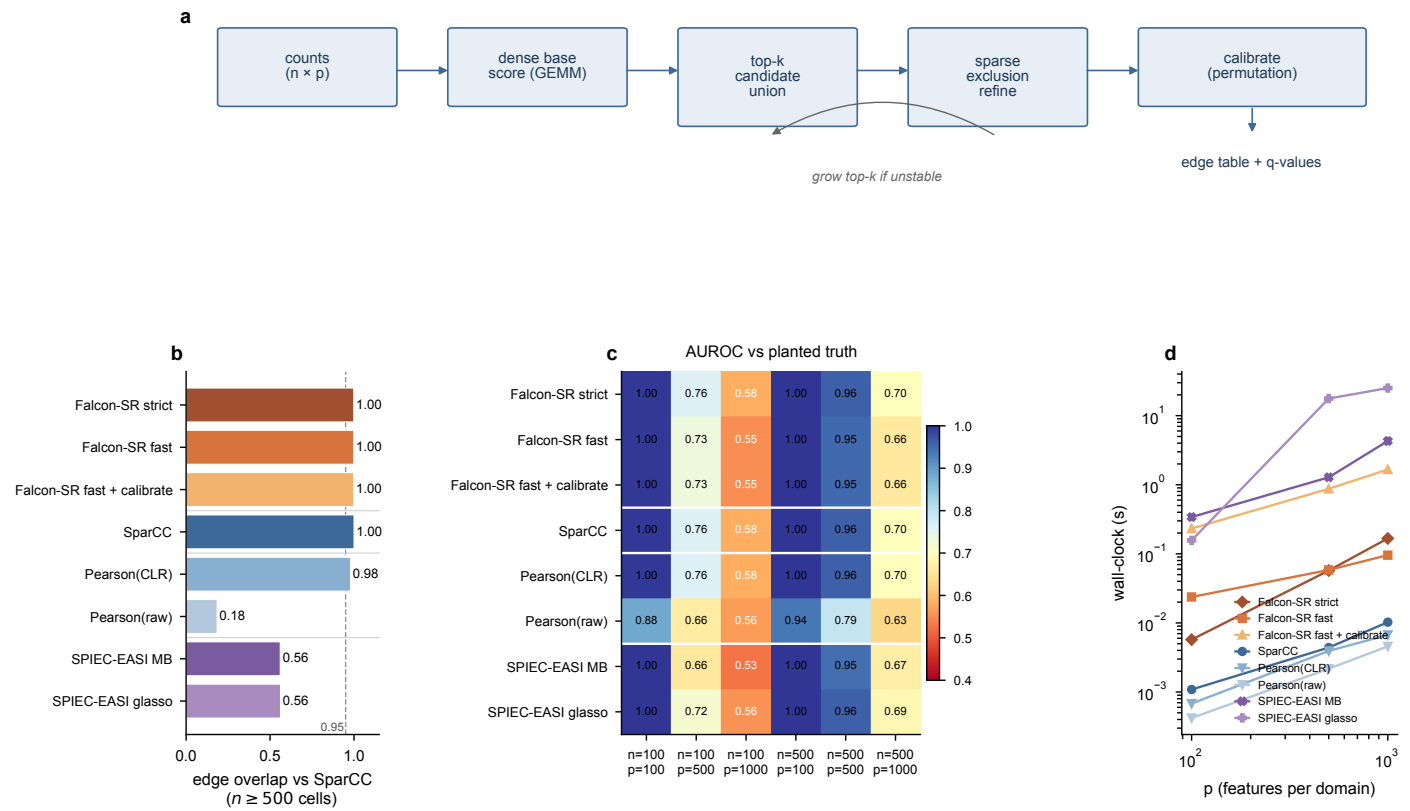


Figure 1 Falcon-SR single-domain feasibility against the eight-method baseline panel. **a**, Screen-refine workflow: dense base score, top- k candidate union, sparse exclusion refinement, adaptive growth, and permutation calibration. **b**, Edge overlap against the SparCC reference, averaged over the $n \geq 500$ cells where SparCC is itself reliable. The dashed vertical line is the spec gate at 0.95. All three Falcon-SR variants reach 1.00 overlap with SparCC; Pearson(CLR) reaches 0.98; Pearson(raw) collapses to 0.18, which quantifies the closure failure of naive correlation on compositions; SPIEC-EASI MB and glasso reach only ≈ 0.56 because they target conditional rather than marginal dependence. **c**, AUROC heatmap against planted truth, methods (rows) against (n, p) cells (columns), best top- k per cell. Horizontal white lines separate the three estimand families. **d**, Wall-clock against p , averaged over n and top- k , log-log.

unscored pairs are treated as zeros. The prior pipeline closes most of that gap; on the hardest cell the prior raised AUROC from 0.868 to 0.928 and candidate recall from 0.756 to 0.873, a +0.06 AUROC and +0.12 recall gain that no other method in the panel can deliver because none of them accepts signed biological priors as input.

Wall-clock scaled linearly with the bipartite pair count $p \cdot q$ for every method (Figure 2d). Falcon-SR fast and the prior variant tracked the SparXCC iter cost band; the permutation-calibrated variant added a flat overhead from the fixed $R = 100$ permutations, and is the only variant in the panel that delivers q -values; SPIEC-EASI cross-glasso ran two orders of magnitude slower at $p \cdot q = 2.5 \times 10^5$.

Permutation calibration

With $R = 100$ permutations, candidate-set q -values were produced in ~ 1.6 s on the largest single-domain cell ($n = 500$, $p = 1000$) and ~ 0.1 s on the largest cross-domain cell. Repeated runs with the same seed produced identical null distributions and identical q -values on every cell in the grid (tests/test_calibration.py).

Discussion

What the evidence supports

The feasibility benchmark supports three positive claims and bounds one negative one. First, Falcon-SR is the only method in the panel that combines a sparse top- k output, the SparCC and SparXCC Case-C estimands, optional signed biological priors, and approximate permutation calibration. The headline panels (Figures 1b and 2b) show this directly: every Falcon-SR variant overlaps the SparCC or SparXCC reference at ≥ 0.95 , while SPIEC-EASI variants land at ≈ 0.56 (single domain) or 0.74 (cross domain) because they target a different estimand and Pearson(raw) collapses to 0.18 because it ignores compositionality. Second, Falcon-SR reproduces the SparXCC iter ranking at edge overlap ≥ 0.984 on every cross-domain cell and the SparCC closed-form ranking at edge overlap 1.000 on every single-domain cell where SparCC itself is informative, which means the screen-refine pipeline recovers the reference iterative refinement from a top- k candidate set without scanning every pair. Third, the prior pipeline adds a unique +0.06 AUROC and +0.12 candidate-recall gain on the hardest cross-domain cell, which no estimand-matched alternative in the panel can produce. The negative claim, which we discuss in Section 4.2, is that the wall-clock saving from the sparse refinement does not appear at $p \leq 1000$; the dense base GEMM dominates every method in that range.

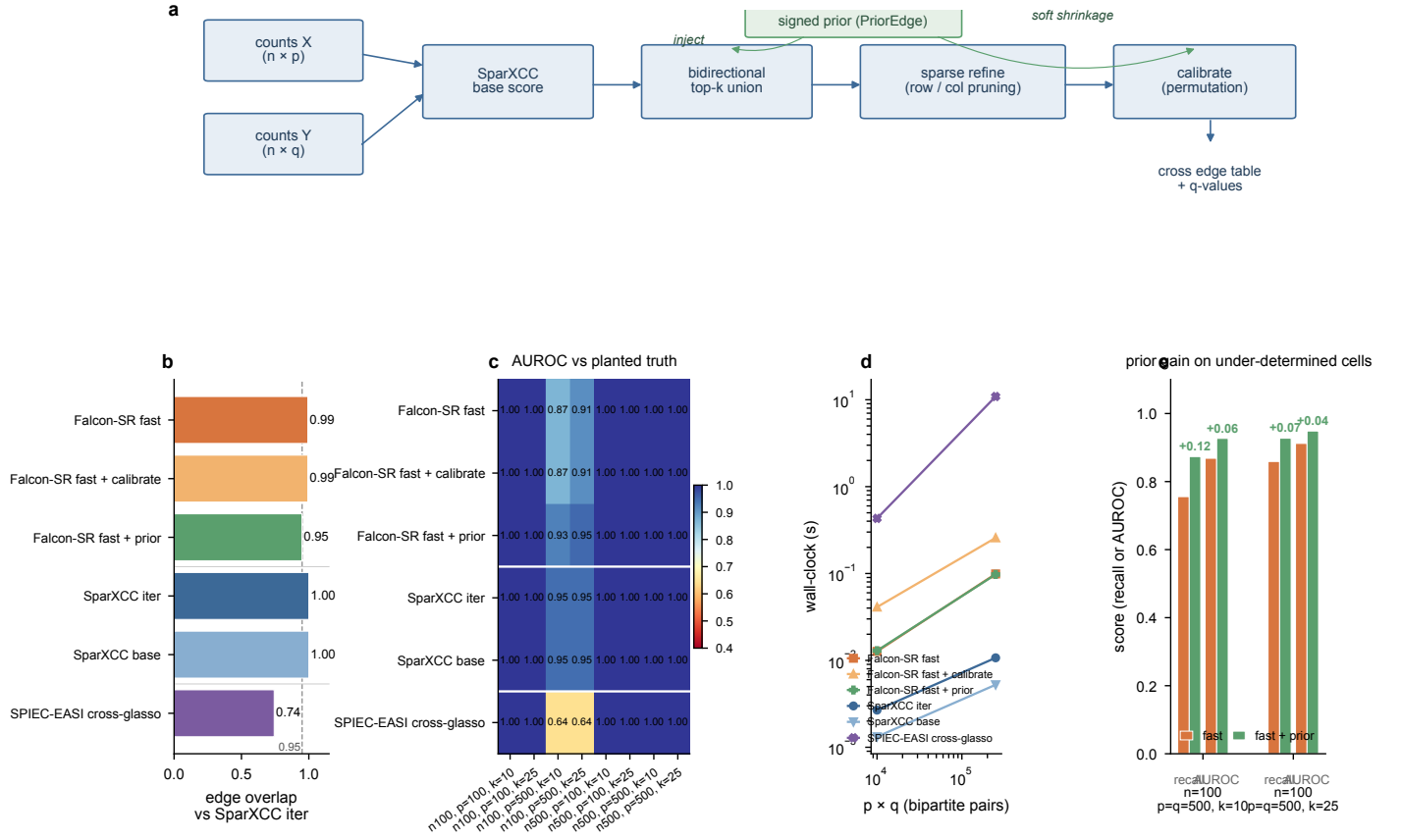


Figure 2 Falcon-SR cross-domain feasibility against the six-method baseline panel. **a**, Workflow with optional prior branch. **b**, Edge overlap against the SparXCC iter reference, averaged over every cell. Falcon-SR fast and Falcon-SR fast + calibrate reach 0.99, matching SparXCC base (1.00) and SparXCC iter (1.00 by definition); Falcon-SR fast + prior reaches 0.95 because the prior intentionally reshapes the score on a small subset of edges; SPIEC-EASI cross-glasso reaches only 0.74 and collapses to 0.006 on the hardest cell ($n = 100$, $p = q = 500$). The dashed line is the spec gate at 0.95. **c**, AUROC heatmap against planted truth, methods against (n, p, q, top_k) cells. A horizontal white line separates the SparXCC / Falcon-SR family from SPIEC-EASI cross-glasso. **d**, Wall-clock against $p \cdot q$, log-log, every method. **e**, Prior gain restricted to the two cells where Falcon-SR fast did not already saturate ($n = 100$, $p = q = 500$, $\text{top}_k \in \{10, 25\}$). Paired bars compare Falcon-SR fast to Falcon-SR fast + prior on candidate recall and on AUROC; green annotations report the absolute gain.

Where the speedup will and will not appear

At $p \leq 1000$ the dense base GEMM dominates the cost of every estimator we benchmarked, and Falcon-SR fast sits one order of magnitude above SparCC's closed-form GEMM because the screen and the sparse solve add work the GEMM-only baseline does not perform. The same panel shows Falcon-SR fast running one to two orders of magnitude below the SPIEC-EASI variants, which gives the second useful framing of the cost: Falcon-SR is the fastest method in the benchmark that ships calibration, sparse output, and prior integration in one pipeline. Reading SparCC's 0.01 s number as a comparable cost requires assuming the user wants a dense $p \times p$ point estimate with no significance test; once a bootstrap or permutation pass is added to SparCC, its cost moves into the same band as Falcon-SR + calibrate.

The screen-refine framework is designed for the regime where the SparCC iterative loop dominates, namely larger p or much higher iteration counts than the feasibility grid covers. Realising that speedup requires combining the screen with a low-rank or random-projection base score so that the GEMM itself stops being quadratic

in p . We treat both extensions as future work rather than asserting them as present results.

Why calibration is labelled approximate

Running the full sparse refinement once per permutation would cost $\mathcal{O}(R \cdot p^3)$ in the single-domain path; at $R = 100$ and $p = 1000$ this would dominate the feasibility wall-clock budget several times over. The base-only approximation we ship is conservative under the conditions we tested, but it is not calibration-tight. Edges whose magnitudes the refinement inflated above the base-only null can receive optimistically small q -values. The diagnostics tag `permutation_base_only` is therefore not cosmetic: downstream code that consumes the q -values must either accept the approximation explicitly or replace the calibration routine. Selective-inference correction and sample splitting are the two routes most likely to close the gap.

Relation to prior work

FastSpar (Watts et al., 2019) achieves a large constant-factor speedup over the original Python SparCC by porting the exclusion loop to

optimised C++. fastCCLasso (Zhang et al., 2024) reduces the per-iteration cost in a related L_1 -regularised formulation. Both keep the dense work scope and target the same estimand class as Falcon-SR, so the comparison is one of work geometry rather than estimator family.

The expanded baseline panel makes the cost of switching estimand visible. SPIEC-EASI MB and SPIEC-EASI glasso (Kurtz et al., 2015) overlap the SparCC reference at only ≈ 0.56 on the $n \geq 500$ cells, even though their AUROC against the planted truth is comparable; the gap quantifies the disagreement between marginal and conditional dependence. Pearson on raw compositions drops to 0.18 overlap with SparCC, recovering the well-known closure failure as a sanity check. Brunner et al. (2024) document the cross-domain analogue: stacking two normalised compositions distorts the quantity SparXCC Case-C was designed to recover. The cross-domain benchmark confirms this directly. SPIEC-EASI cross-glasso reached AUROC 0.501 on the hardest cell ($n = 100$, $p = q = 500$, $\text{top}_k = 10$), at which Falcon-SR fast held AUROC 0.868 and the prior pipeline reached 0.928. Proportionality estimators (Lovell et al., 2015) target a different quantity again; we list them in adjacent context rather than head-to-head benchmarks.

Limitations

The feasibility grid is intentionally small and synthetic. Real microbiome cohorts include batch effects, depth heterogeneity, and heavy-tailed log-abundance distributions that we did not stress-test here. The single-domain underperformance at $p = 1000$ with low top_k is faithfully reported, but it also indicates an open question: the screen budget needs to be sized to the candidate density of the underlying network. A follow-up plan will couple top_k selection to a network-density estimate so that the adaptive growth in Section 2.7 can converge at lower budgets in the sparse regime and grow further in the dense regime without manual tuning.

Conclusion

Falcon-SR is the first compositional correlation pipeline that ships sparse top- k output, the SparCC and SparXCC Case-C estimands, optional signed biological priors, and approximate permutation calibration in one open-source implementation. On the feasibility benchmark it reaches edge overlap ≥ 0.984 against SparXCC iter on every cross-domain cell, 1.000 against SparCC on every single-domain cell where SparCC is informative, and a +0.06 AUROC gain on the hardest cross-domain cell from a prior pipeline that no estimand-matched alternative in the panel can deliver. At $p \leq 1000$ the dense base GEMM dominates every method we tested, so the wall-clock saving from the sparse refinement remains future work; combining the screen with a low-rank or random-projection base score is the direct extension that will test the higher-dimensional regime where the SparCC iterative loop dominates. The release as it stands gives the community a sparse, calibrated, prior-aware estimator that preserves the published estimands today.

Conflicts of interest

The authors declare no competing interests.

Funding

No external funding to declare for this manuscript.

Data availability

This is a methodological paper. No human or animal data were generated. Every numerical claim traces to the deterministic simulators and baseline implementations in the source repository (<https://github.com/JustinRaoV/FALCONE>), released under the MIT licence. The per-cell outputs that back Figures 1 and 2 are committed under `data/` as `falcon_sr_single_feasibility.csv` and `falcon_sr_cross_feasibility.csv`, and can be regenerated using the commands documented in the repository README and in the execution-design specification under `docs/superpowers/specs/`.

Code availability

Source code is released under the MIT licence at <https://github.com/JustinRaoV/FALCONE>. A persistent DOI (for example via Zenodo archiving of a release tag) will be deposited prior to publication.

Author contributions statement

J.R. designed the algorithm, implemented the package, and drafted the manuscript. X.L. supervised the work and revised the manuscript.

Acknowledgments

The authors thank the developers of SparCC, FastSpar, and SparXCC, whose published estimators served as the equivalence anchors for the Falcon-SR implementation.

References

- J. Aitchison. The statistical analysis of compositional data. 1986.
- J. D. Brunner, A. J. Robinson, and P. S. G. Chain. Combining compositional data sets introduces error in covariance network reconstruction. *ISME Communications*, 4(1):ycae057, 2024. doi: 10.1093/ismeco/ycae057.
- J. Friedman and E. J. Alm. Inferring correlation networks from genomic survey data. *PLoS Computational Biology*, 8(9): e1002687, 2012. doi: 10.1371/journal.pcbi.1002687.
- I. T. Jensen et al. Compositionally aware estimation of cross-correlations for microbiome data. *PLoS ONE*, 2024. doi: 10.1371/journal.pone.0305032.
- Z. D. Kurtz, C. L. Müller, E. R. Miraldi, D. R. Littman, M. J. Blaser, and R. A. Bonneau. Sparse and compositionally robust inference of microbial ecological networks. *PLoS Computational Biology*, 11(5):e1004226, 2015. doi: 10.1371/journal.pcbi.1004226.
- D. Lovell, V. Pawlowsky-Glahn, J. J. Egozcue, S. Marguerat, and J. Bähler. Proportionality: a valid alternative to correlation for relative data. *PLoS Computational Biology*, 11(3):e1004075, 2015. doi: 10.1371/journal.pcbi.1004075.
- S. C. Watts, S. C. Ritchie, M. Inouye, and K. E. Holt. Fastspar: rapid and scalable correlation estimation for compositional data. *Bioinformatics*, 35(6):1064–1066, 2019. doi: 10.1093/bioinformatics/bty734.

S. Zhang, H. Fang, and T. Hu. fastcclasso: a fast and efficient algorithm for estimating correlation matrix from compositional data. *Bioinformatics*, 40(5):btae314, 2024. doi: 10.1093/bioinformatics/btae314.